

A Knot-based Key Exchange protocol

Silvia Sconza* and Arno Wildi†

University of Zurich, Zurich, Switzerland

Abstract

We propose a new key exchange protocol based on the Generalised Diffie-Hellman Key Exchange. In the latter, instead of using a group action, we consider a semigroup action. In our proposal, the semigroup is the set of oriented knots in $\mathbb{S}^3$ with the operation of connected sum. As a semigroup action, we choose the action of the semigroup on itself through the connected sum. For the protocol to work, we need to use knot invariants, which allow us to create the shared secret key starting from the same knot represented in two different ways. In particular, we use finite type invariants. The security of the protocol is guaranteed by the hardness of decomposing knots in the semigroup.

Keywords: Diffie-Hellman Key Exchange, Public Key Cryptography, Knot Theory, Finite Type Invariants, Semigroup-Based Cryptography.

1 Introduction

Knots, which are ambient isotopy types of embeddings $\mathbb{S}^1 \hookrightarrow \mathbb{S}^3$ (see Figure 2 and Definition 2.1), were used by humankind since ancient times, at the latest since the invention of the shoelace. The mathematical study of knots however started with Lord Kelvin, hypothesizing that atoms are actually knots and molecules are links flowing in the aether. His collaborator Peter Tait then initiated the field of knot theory. The basic problem being: Given two knots, are they the same or not? Following the development of topology in the early 20th century, numerous knot invariants like the Alexander polynomial [2] were developed to give answers to this problem. The interest in knot theory rose when deep connections were found to the study of 3- and 4-manifolds. For example, knots were used to prove that there are exotic $\mathbb{R}^4$, i.e. manifolds that are homeomorphic but not diffeomorphic to $\mathbb{R}^4$ [18]. Jones and Witten revolutionized the field with the discovery of the Jones polynomial [23] and its relation to Quantum field theory [44] from which quantum topology emerged. These breakthroughs were followed by the discovery of Khovanov homology [25] and knot Floer homology [38], which vastly generalize the Jones and the Alexander polynomial and provide active fields of research.

In this paper we are mainly interested in two aspects of knot theory. The first is an operation called the connected sum (see Figure 5) that takes two oriented knots, cuts them open and glues

*silvia.sconza@math.uzh.ch

†arno.wildi@math.uzh.ch

them together, respecting the orientation, to produce a new oriented knot. It turns out that, with this operation, oriented knots form the abelian semigroup **Knots**. The second is a class of knot invariants (see Section 2.1.3) called finite type invariants. They have many interesting features, one of them being that they are efficiently computable.

In the last 30 years, applications of knot theory to other scientific disciplines were found. In chemistry, we can decide whether a molecule is chiral or not. Molecules can have very different properties depending on their chirality. In biology, one can study the knottedness of the DNA in a cell. And in quantum computing, one studies anyons which give naturally a braid that can be studied in knot-theoretic terms.

In the paper, we propose an application of knot theory in cryptography, in particular we propose a Key Exchange Protocol. We know of three instances where knots were considered in cryptography. After getting the Fields Medal, Jones was asked whether knotted antennas would help sending messages securely. Together with Przytycki they investigated the problem. More promising attempts were the protocol of Marzuoli and Palumbo in [31] and the secret-key agreement proposed by Zucker in [45]. In the first one, they are proposing a symmetric protocol, the weakness of which is that they have to agree in secret on as much information as is used to describe the message. The second one is more close to our proposal since it is also a Key Exchange Protocol, but the author is using knots in braid form (see Subsection 4.1.2). Moreover, the use of the Jones Polynomial allows us to break it, since it is multiplicative. We will discuss in Section 4 why it is necessary to avoid such invariants.

In cryptography, we are concerned with exchanging messages securely, i.e. in such a way that an eavesdropper outside the conversation cannot obtain the original message. To do this, the message is “reformed” using a cryptographic key and only the designated recipient can retrieve the original message again using a key (not necessarily the same one). One of the main problems facing cryptography is how to generate and exchange these keys (Key Exchange Problem). One of the most widely used ways is the Diffie-Hellman Key Exchange, proposed in [16] and described below in Protocol 2.20. The original protocol works for a finite cyclic group G (in the first proposal $G = \mathbb{F}_q^\times$), but it can be generalised using a group action or a semigroup action, obtaining the Generalised Diffie-Hellman Key Exchange, described below in Protocol 2.24. In our case, the semigroup that we consider is **Knots**. The semigroup action is the action of the semigroup on itself, through the connected sum.

In the past, several cryptosystems based on group-actions have been proposed. The most famous are the one proposed by Anshel, Anshel and Goldfeld [4] and Ko et al. [26], where the groups considered are the braid groups and the action is the conjugation, and the one proposed by Castryck et al. [13], known as CSIDH (Commutative Supersingular Isogeny Diffie-Hellman). The latter is part of Isogeny-based Cryptography; the set considered is that of $\mathbb{F}_p$ -isomorphism classes of supersingular elliptic curves over $\mathbb{F}_p$, characterised by the fact that the ring of endomorphisms is an order $\mathcal{O}$ in an imaginary quadratic field. The acting group is the ideal-class group $\text{cl}(\mathcal{O})$ which acts through the application of isogenies.

As we will see also in our case, the use of semigroup actions instead of group actions brings certain advantages. First of all, given the poorer algebraic structure, many attacks cannot be applied or generalised. Generic algorithms for the group case like Pollard’s rho [40], Pollard’s lambda [40] and Shank’s baby-step-giant-step [43] require at some point the existence of inverses, which is not ensured in the case of semigroups.

The choice of considering **Knots** and the semigroup action on itself brings certain advantages. First of all, referring to the aforementioned attacks, no knot admits an inverse except for the trivial

one, i.e. the unknot $\mathcal{U}$, as stated in Proposition 2.6. Furthermore, in order to propose a well-defined cryptosystem, one must find a mathematical problem which is supposed to be computationally hard. Our cryptosystem is based on the difficulty of factoring a connected sum of knots while knowing one of the two knots used (Problem 2.14). The particularity of this problem is that it admits a unique solution, thanks to Proposition 2.7.

The paper is organised as follows. In Section 2 we give all the necessary preliminaries. It is divided in two subsections: in the first one, we give the preliminaries of Knot Theory and in the second one, there are the preliminaries related to Public-Key Cryptography. In Section 3, we describe our proposed new cryptosystem, with an indication of the size of the keys. Section 4 is dedicated to the cryptoanalysis; there we analyse both generic attacks for the semigroup action problem and knot theoretic attacks. Moreover, we briefly outline the efficiency. Finally, we discuss the choice of parameters and we conclude with some open questions. The security proof against passive adversaries can be found in Appendix 2.

Acknowledgement. The authors would like to thank their respective advisors Joachim Rosenthal and Anna Beliakova for their generous support and their helpful suggestions. We would like to thank Léo Ducas for useful comments on a first draft of the paper. We are also extremely grateful to Chris Monico, who is working on an implementation of our cryptosystem and for useful comments that allowed us to obtain the final version of the paper, highlighting some problems with the first proposed obfuscation algorithm. It is also his idea to use a hash function and to compute not just one, but several finite type invariants of small degree. The second author would like to thank Peter Feller, Józef Przytycki and Daniel Tubbenhauer for interesting conversations on the use of knot invariants in cryptography. The first author gratefully wishes to note: this research is supported by armasuisse Science and Technology.

2 Preliminaries

In this section we introduce the required topics for understanding the proposed cryptosystem. A reader coming from the knot-theory-side may skip the part on knots (although one might want to refresh the memory on finite type invariants from Section 2.1.3), while a reader coming from the cryptography-side may skip the part on cryptography.

2.1 Crash course on knots

We discuss the relevant notions and results from knot theory, but leave out most of the proofs. For a general introduction to knot theory we refer to [42].

2.1.1 Basic notions and problems from knot theory

Intuitively, knots are exactly what one expects them to be. We think of them as tied ropes where we glue the ends together and allow the rope to “wobble”. Let us start by giving a mathematically precise definition.

Definition 2.1. A *knot* is described by either of the following (equivalent) definitions.

1. Knots are smooth embeddings $\mathbb{S}^1 \hookrightarrow \mathbb{R}^3$, considered up to ambient isotopy. ¹
2. Knots are (finite) polygonal closed lines in $\mathbb{R}^3$, considered up to the Δ -move seen in Figure 1.

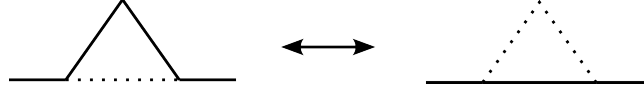


Figure 1: A local picture of the Δ -move.

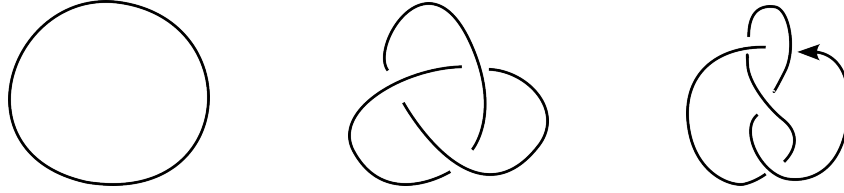


Figure 2: Some examples of knots (diagrams). The unknot $\mathcal{U}$, the (right-handed) Trefoil knot and the oriented Figure-Eight knot.

If one allows several components, one speaks of a *link*. Usually, we picture knots by *knot diagrams*. Some examples can be seen in Figure 2. They are generic projections of the knot to a plane allowing only singularities of a certain kind, called *crossings*. The complexity of a knot diagram can be measured by its number of crossings. Of course, a knot has many different diagrams, depending on its position in $\mathbb{R}^3$ and the chosen projection. The *Reidemeister theorem* allows us to handle this problem.

Theorem 2.2 (Reidemeister). *Two knot diagrams represent the same knot if and only if they are related by planar isotopies and a finite sequence of the Reidemeister moves, represented in Figure 3.*

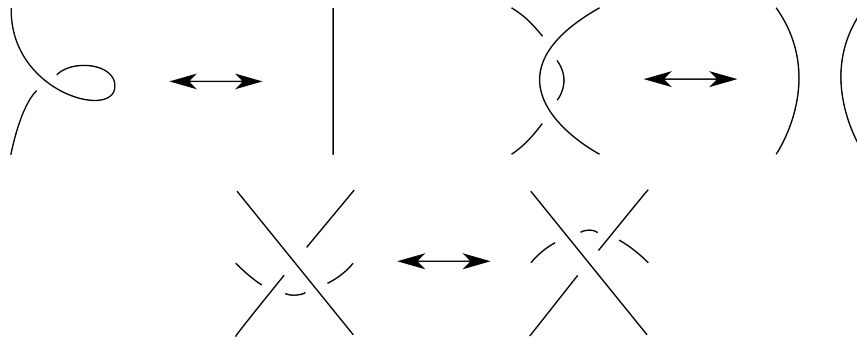


Figure 3: The three Reidemeister moves for knot diagrams.

¹Two embeddings $g, h: N \hookrightarrow M$ are called ambient isotopic, if there is a continuous map (called ambient isotopy) $F: M \times [0, 1] \rightarrow M$ such that $F_0 = id_M$ and $F_1 \circ g = h$.

Definition 2.3. When a knot is endowed with an orientation, it is called an *oriented knot*. There is a corresponding Reidemeister theorem for oriented knots. We distinguish two types of crossings called the *positive* and the *negative* crossing seen in Figure 4. The set of oriented knots is denoted by **Knots**.

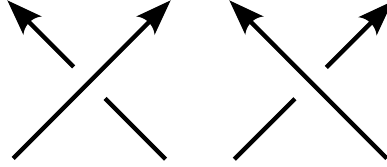


Figure 4: The positive and the negative crossing.

We consider the following operation on oriented knots.

Definition 2.4. Given two oriented knots K and K' we define the *connected sum* $K \# K'$ by cutting open the two knots and glueing the corresponding ends (given by the orientation) together as in Figure 5 to get a new knot.

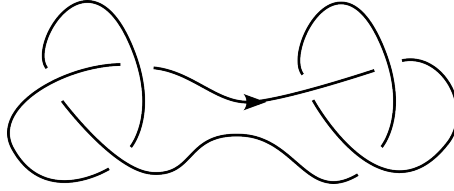


Figure 5: The connected sum of two Trefoil knots.

The connected sum of two knots is well defined, because cutting the knots at different spots results in isotopic knots. Note that it is not well defined for unoriented knots, since we have a choice, i.e. which arcs we glue together in the unoriented case. It is obvious that taking the connected sum with the unknot $\mathcal{U}$ yields an isotopic knot. The following Proposition is immediate.

Proposition 2.5. $(\mathbf{Knots}, \#, \mathcal{U})$ form an abelian semigroup.

We call a knot that cannot be written in a non-trivial way as the connected sum of two other knots *prime*. The semigroup of oriented knots has the following properties proven in [42].

Proposition 2.6. *Apart from the unknot, no knot has an inverse with respect to the connected sum.*

Proposition 2.7. *For any knot there is a unique (up to reordering) decomposition into prime knots.*

In general, it is hard to tell whether a knot is prime or not. But for a certain subset of knots, it is straightforward.

Definition 2.8. A knot diagram is called *alternating*, if the crossings alternate between under- and overpasses when travelling along the knot. A knot is called *alternating*, if it has an alternating diagram.

Not all knots are alternating. There are even prime knots which are not alternating. There are the following useful fact about alternating knots proven in [34] and [36].

Proposition 2.9. *An alternating knot diagram represents a prime knot if and only if the diagram “looks prime”, meaning that there is no circle that intersects the diagram at two points such that there are crossings both inside and outside the circle. Moreover, such an alternating prime diagram is minimal, i.e. there is no diagram of the knot with less crossings.*

This gives a method to generate random prime knots. With Schaeffer’s algorithm, one can generate a random planar graph with n vertices in $\mathcal{O}(n)$ time. This will give a graph representing a link in general, but one can take the biggest component which is a knot. There is a unique (up to mirroring) way of making the diagram alternating. Then, one can take the biggest prime summand and this will give in practice a prime knot with $\frac{n}{2}$ to n crossings. This is implemented in the “SnapPy” program. An example of a randomly produced knot is seen in Figure 6.

A question to ask is, whether one gets enough knots this way, or if the set of alternating prime knots is comparably small. While Table 1 provides the number of small (alternating) prime knots, the following estimate, proven in [1], answers the previous question in our favour. In Proposition 2.10 we will refer to knots that admit a minimal representation with at most n crossings as *minimal* knots with at most n crossings.

n	1	2	3	4	5	6	7	8	9	10	11	12
# n crossings prime knots	0	0	1	1	2	3	7	21	49	165	552	2176
# n crossings alternating prime knots	0	0	1	1	2	3	7	18	41	123	367	1288

13	14	15	16	17	18	19	20	21
9988	46972	253293	1388705	8053393	48266466	294130458	unknown	unknown
4878	19536	85263	379799	1769979	8400285	40619385	199631989	990623857

Table 1: The list of the number of (alternating) prime knots with n crossings.

Proposition 2.10. *Let $K(n)$ be the set of minimal knots with at most n crossings, $PK(n)$ be the set of prime minimal knots with at most n crossings, and $APK(n)$ be the set of alternating prime minimal knots with at most n crossings. We have the following estimate*

$$4.45 \leq \liminf |APK(n)|^{\frac{1}{n}} \leq \liminf |PK(n)|^{\frac{1}{n}} \leq \liminf |K(n)|^{\frac{1}{n}} \leq 10.40.$$

In cryptography, one is usually interested in problems that are hard to solve. Topology is full of such problems and, in particular, we have examples in knot theory.

Problem 2.11 (Unknotting Problem). Given a diagram D , does it represent the unknot?

Problem 2.12 (Recognition Problem). Given two knot diagrams D and D' , do they represent the same knot?

Problem 2.13. Given a knot diagram D of a knot K , are there non-trivial knots K_1 and K_2 such that $K_1 \# K_2 = K$? Can you find (diagrams of) K_1 and K_2 ?

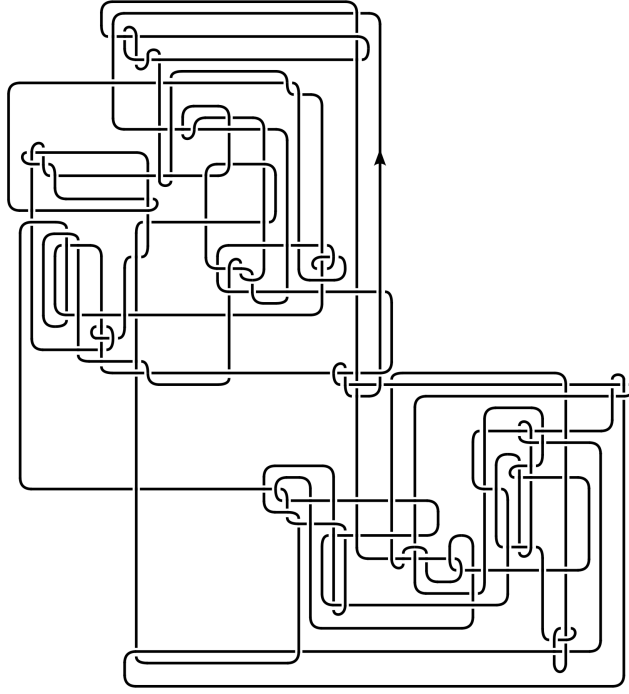


Figure 6: A randomly produced alternating prime knot with 151 crossings.

Problem 2.14 (Division Problem). Let D be a knot diagram of a knot $K = K_1 \# K_2$ and assume you know (a diagram of) K_1 . Find (a diagram of) K_2 .

It was shown in [21] that Problem 2.11 is in NP (which stands for Nondeterministic Polynomial time). It is the set of decision problems verifiable in polynomial time by a deterministic Turing machine. So far the best unknotting algorithms are exponential ([11], [12], [28], [17]). In [17], it was shown that Problem 2.13 and Problem 2.14 are in NP . However, the presented algorithm is very ineffective for knots with many crossings. The hardness of Problem 2.14 is what we use to our advantage in the construction of the cryptosystem.

2.1.2 Encoding knots

The following part is dedicated to transferring the cryptosystem into the world of computers. Since we are using knots, we need to have some way of encoding these topological objects. We will introduce the so called *PD* (Planar Diagram) *notation*.

PD notation. Consider a knot diagram D . We can see it as a 4-regular graph with the additional information about which strand goes over which one at every vertex. The goal now is to encode this information efficiently. Choose a starting point on an arc of the knot and the direction along the knot, given by the orientation. Start by labelling this arc with 1. The next will get labelled with 2 and so on until every arc has a number associated to it. Now, we associate to each crossing a quatern with the labels of the four arcs attached to it: we start with the label of the unique incoming

undergoing arc and we conclude with the other three in counter-clockwise order. At the end, form a list with all these quaterns. This holds the information about the underlying 4-regular graph. An example is seen in Figure 7.

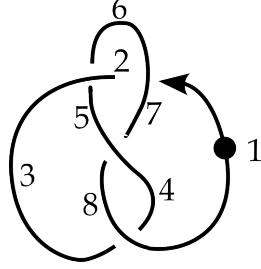


Figure 7: The PD notation of the Figure Eight knot pictured above is the following list: $[(1, 6, 2, 7), (5, 2, 6, 3), (3, 1, 4, 8), (7, 5, 8, 4)]$.

PD notation and connected sum. Notice that the PD notation in Figure 8 for the connected sum of twice the Figure Eight knot splits into two sublists where in the first one there are all the numbers between 1 and 9 and in the second one there are the the remaining ones from 9 over 16 to 1. This fact is true in general for the code of the connected sum, if the starting point for the code lies on one of the connecting arcs. If the starting point lies somewhere else, we can get it to lie on the connecting arc by shifting every number by a fixed shift modulo twice the number of crossings.

To perform the connected sum of two knots we can use the previous observation to split the first arc of the two knots and combine the list of tuples, shifting the numbers of the second list.

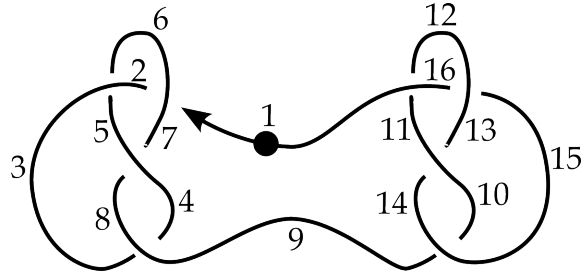


Figure 8: The PD notation for the connected sum of two Figure Eight knots is $[(1, 6, 2, 7), (5, 2, 6, 3), (3, 9, 4, 8), (7, 5, 8, 4), (9, 15, 10, 14), (13, 11, 14, 10), (11, 16, 12, 1), (15, 12, 16, 13)]$.

PD notation and Reidemeister moves. We analyze how Reidemeister moves performed on the diagram change the PD notation. A visualization is seen in Figure 9. There are several possibilities for each move, depending on the orientation of the arcs and the signs of the crossings. In the following we will show one case per move (see also Figure 9).

- Reidemeister 1+ (R1p): It adds a crossing with tuple $(i, i + 1, i + 1, i + 2)$.
- Reidemeister 1- (R1m): It deletes a crossing with tuple $(i, i + 1, i + 1, i + 2)$.

- Reidemeister 2+ (R2p): It adds two crossings between arcs belonging to the same region. The crossings are of the form $(i, j, i+1, j+1)$ and $(i+1, j+2, i+2, j+1)$.
- Reidemeister 2- (R2m): It deletes two crossings of the form $(i, j, i+1, j+1)$ and $(i+1, j+2, i+2, j+1)$.
- Reidemeister 3 (R3): This move changes three crossings $(j, i, j+1, i+1)$, $(j+1, k+2, j+2, k+1)$, $(i+1, k+1, i+2, k)$ to $(i, k+2, i+1, k+1)$, $(j+1, i+1, j+2, i+2)$, $(j, k+1, j+1, k)$.

This gives us the tools to apply random Reidemeister moves to a knot diagram using the PD notation, because we can see from the code where we can do which move. For this, we use a weight system that assigns weights to (R1p, R1m, R2p, R2m, R3) based on which we randomly determine the type of move to perform.

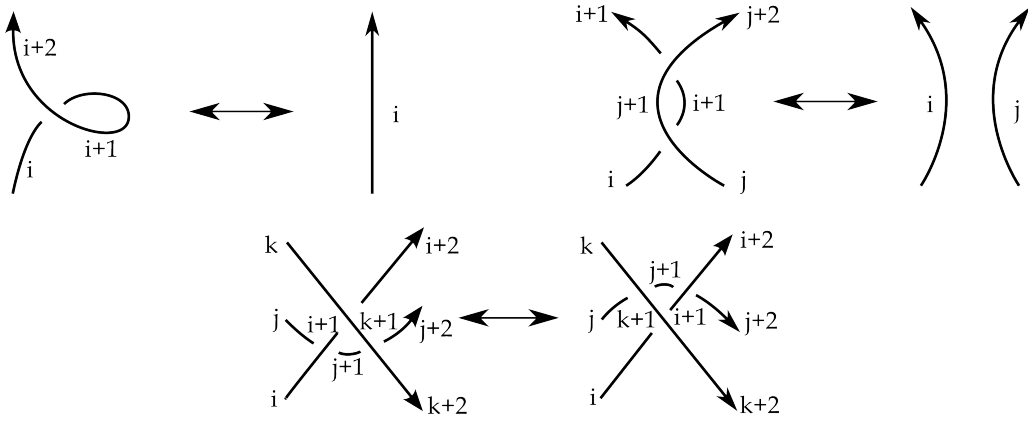


Figure 9: A visualization of how the PD notation transforms under the Reidemeister moves.

PD notation and coding. It is important to note that PD notation is supported by “SnapPy” and the Mathematica package “KnotTheory” from Bar Natan and Morrison [7].

Complexifying knots. We want to investigate ways of bringing a knot into a general, i.e. unrecognisable, position. Usually, one is interested in the converse problem, so our ideas here (except Move I) are, at least to our knowledge, new. To complexify knots, one can do a combination of the following four moves.

- **Move I:** Apply random Reidemeister moves.
- **Move II:** Take a knot diagram and map it into $\mathbb{R}^3$ by taking a random piecewise linear height function, respecting the height order at the crossings. Turn the knot in $\mathbb{R}^3$ by a random element in $SO(3)$ and project to a diagram. This is visualised in Figure 10.
- **Move III:** Take an arc in the diagram and move it around the whole knot such that it passes once underneath the knot. This is visualised in Figure 11.
- **Move IV:** Take an arc in the diagram and move it in an alternating fashion through regions of the diagram. This is visualised in Figure 12.

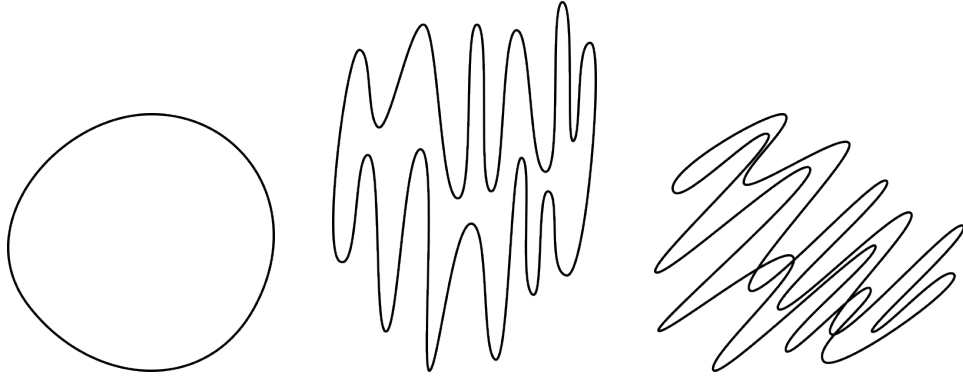


Figure 10: A sketch of the complexification move II. A diagram followed by its corresponding knot with a random height function. The turned knots is visualised at last.

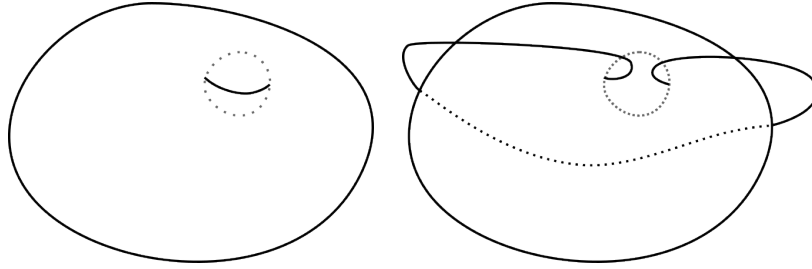


Figure 11: A visualisation of the complexification move III.

Complexifying the knots allows us to obfuscate them. The purpose of this will become clear later, when we introduce our protocol.

2.1.3 Finite type invariants

To classify knots we use functions defined on diagrams that do not change under planar isotopies and Reidemeister moves. These functions are called knot invariants. Prominent examples include the knot signature [36], the Jones polynomial [23], the Alexander polynomial [2] and the finite type invariants studied in this section. Note that these invariants usually do not distinguish all knots and it is common to trade calculability of the invariant for its power to detect knots.

We now consider finite type invariants. An introduction can be found in [5]. Let V (for Vassiliev invariant) be an invariant of knots with values in $\mathbb{Z}$, i.e. a function

$$V: \{\text{Knot Diagrams}\} \rightarrow \mathbb{Z}$$

that does not change under Reidemeister moves. Extend V to an invariant of oriented 1-singular knots, which are knots that have a singularity locally looking like $\times$, using the (local) formula

$$V(\times) = V(\nearrow) - V(\searrow). \quad (1)$$

Further extend V to the set of oriented m -singular knots by using (1) repeatedly.

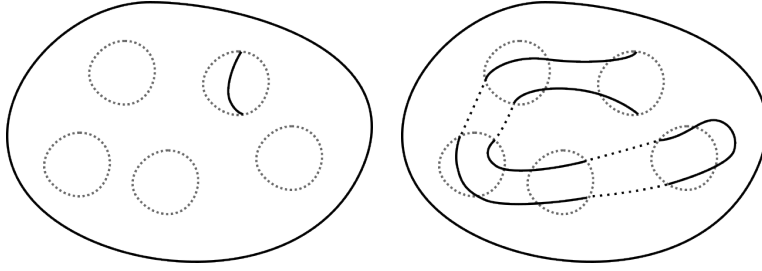


Figure 12: A visualisation of the complexification move IV.

Definition 2.15. We say that V is a *finite type invariant* (of degree m), if its extension to oriented $(m + 1)$ -singular knots vanishes.

Example 2.16 (Conway polynomial). The (Alexander-)Conway polynomial Δ [14] provides a source of finite type invariants. It is a polynomial (in the formal variable z) knot invariant, with value 1 on the unknot, satisfying the following local (skein) relation

$$\Delta(\nearrow \searrow) - \Delta(\nwarrow \swarrow) = z \cdot \Delta(\searrow \swarrow)$$

It is immediate from the definition that the coefficient of z^m in the Conway polynomial is a finite type invariant of degree m .

In some sense, finite type invariants forget a lot of data. They carry a “finite” amount of information in the sense of Theorem A.3. Nevertheless there is the following conjecture.

Conjecture 2.17. The set of all finite type invariants distinguishes knots.

The following property of finite type invariants is crucial for the purpose of our paper. We postpone the proof to Appendix A.

Corollary 2.18. A finite type invariant of degree m can be computed in polynomial time $\mathcal{O}(c^m)$, depending on the number of crossings c .

The proof will give an explicit way of constructing any finite type invariant. Parts of this are already encoded in Bar-Natan’s, Amir-Khosravi’s, and Sankarans Program “VasCalc” [3]. In Table 2, we see the list of the number of linearly independent finite type invariants.

m	0	1	2	3	4	5	6	7	8	9	10	11	12
#Degree m finite type invariants	1	1	2	3	6	10	19	33	60	104	184	316	548

Table 2: The list of the number of linearly independent finite type invariants of a given degree m .

Recently, it was shown in [6] that one can compute finite type invariants significantly faster, as stated in the Theorem below.

Theorem 2.19. A finite type invariant of degree m can be computed in polynomial time $\mathcal{O}(c^{\frac{m}{2}})$, depending on the number of crossings c .

Note that finite type invariants are supported in the “KnotTheory” package of Mathematica [7].

2.2 Crash course on cryptography

The aim of this section is to introduce the notions of cryptography, particularly relating to key exchanges, which we will use in the main part of the article.

The purpose of cryptography is to find ways (protocols) to communicate securely, assuming the presence of eavesdroppers. In particular, we want to transform our messages (*encryption phase*) in such a way that opponents will find it to be unintelligible text and only the predestined receiver will be able to trace the original message (*decryption phase*).

In order to carry out encryption and decryption, we need so-called *cryptographic keys*. This brings us to the *Key Exchange Problem*: how can two parties exchange keys in such a way as to establish a secure communication channel? This process is called a *key exchange protocol* and, in the next section, we will propose one based on knots.

There are two main types of cryptography: *symmetric-key* cryptography (or *single-key* cryptography) and *asymmetrical* cryptography (or *public-key* cryptography). In the first one, the two parties (Alice and Bob) use the same secret key to encrypt and decrypt the message. The main problem in this case is that they need to communicate this key to each other via a secure channel. The commonly used symmetric cryptosystem is the Advanced Encryption Standard (AES) [15, 22].

In public-key cryptography, there are two keys involved: a public one known to everybody and a private one known only to the owner; depending on the algorithm, one will be used to encrypt and the other to decrypt. One of the earliest and most famous examples of Public-Key Exchange (PKE) is the *Diffie-Hellman Key Exchange* [16], proposed by Diffie and Hellman in 1976 and described below in Protocol 2.20. It allows the two parties to establish a shared secret key over an insecure channel, i.e. even in the presence of eavesdroppers who can monitor the channel.

Protocol 2.20 (Diffie-Hellman Key Exchange).

1. Alice and Bob publicly agree on a cyclic finite group G and a generator g .
2. Alice chooses $a \in \{1, \dots, \text{ord}(G)\}$, computes g^a and sends it to Bob. Her secret key is a .
3. Bob chooses $b \in \{1, \dots, \text{ord}(G)\}$, computes g^b and sends it to Alice. His secret key is b .
4. Alice computes $(g^b)^a = g^{ba}$.
5. Bob computes $(g^a)^b = g^{ab}$.

The secret common key is $g^{ba} = g^{ab}$.

The security of the system is based on the following hard problem:

Problem 2.21 (Diffie-Hellman Problem (DHP)). Let G be a finite cyclic group and let g be a generator. Given g^a and g^b , find g^{ab} .

We will say that a mathematical problem is *easy* if there exists a polynomial-time algorithm which can solve it. If there are no deterministic or probabilistic polynomial-time algorithms that can solve it, we will call it *hard*.

Actually, if someone is able to solve the Discrete Logarithm Problem (DLP): *given G an abelian group and $a, b \in G$ such that $b = a^m$ for some $m \in \mathbb{Z}$, find $0 \leq n < \text{ord}(G)$ such that $a^n = b$* , they can also solve the DHP.

Remark 2.22. In a cryptosystem, we need the computations required for implementation to be feasible, and those needed to break it to be not. In the Diffie-Hellman Key Exchange 2.20, we have that g^a can be computed in $\mathcal{O}(\log a)$ group multiplications, while the best algorithm to solve the DLP requires $\mathcal{O}(\sqrt{\text{ord}(G)})$.

Notice that we define the Diffie-Hellman protocol using the group action of $\mathbb{Z}_{\text{ord}(G)}^\times$ over G given by

$$\begin{aligned}\mathbb{Z}_{\text{ord}(G)}^\times \times G &\rightarrow G \\ (n, g) &\mapsto g^n.\end{aligned}$$

Therefore it is possible to naturally extend the Diffie-Hellman protocol from a generic group action and even from a semigroup action, as shown in [35].

Definition 2.23. Let G be a semigroup and let S be a set. The semigroup G *acts on* S if there exists a map

$$\begin{aligned}G \times S &\rightarrow S \\ (g, s) &\mapsto g \cdot s,\end{aligned}$$

satisfying $(gh) \cdot s = g \cdot (h \cdot s)$ for all $g, h \in G$ and all $s \in S$.

If the semigroup is abelian, the map is called a G -*action* on the set S .

We can then define the *Generalised Diffie-Hellman Key Exchange*.

Protocol 2.24 (Generalised Diffie-Hellman Key Exchange).

1. Alice and Bob publicly agree on a G -action on a finite set S and an element $s \in S$.
2. Alice chooses $a \in G$, computes $a \cdot s$ and sends it to Bob. Her secret key is a .
3. Bob chooses $b \in G$, computes $b \cdot s$ and sends it to Alice. His secret key is b .
4. Alice computes $a \cdot (b \cdot s)$.
5. Bob computes $b \cdot (a \cdot s)$.

The secret common key is $a \cdot (b \cdot s) = (ab) \cdot s = (ba) \cdot s = b \cdot (a \cdot s)$.

In order to obtain a secure cryptosystem, we need to choose an action that makes the following mathematical problem hard.

Problem 2.25 (Diffie-Hellman Semigroup Action Problem (DHSAP)). Let G be an abelian semigroup, S a finite set and $\cdot$ a G -action on S . Given $x, y, z \in S$ such that $y = g \cdot x$ and $z = h \cdot x$ for some $g, h \in G$, find $(gh) \cdot x$.

Notice that, on the other hand, in order to be able to calculate the key, we need the action to be computationally feasible, i.e. we need to be able to easily calculate every $g \cdot s$ and also every multiplication in G .

There is a generalised version of the DLP which use a generic semigroup action, the so-called Semigroup Action Problem.

Problem 2.26. (Semigroup Action Problem (SAP)) Let G be an abelian semigroup, S a finite set and $\cdot$ a G -action on S . Given $x, y \in S$ such that $y = g \cdot x$ for some $g \in G$, find $h \in G$ such that $y = h \cdot x$.

As with the DLP and the DHP, if one is able to solve the SAP, then he can automatically solve the DHSAP; indeed, let $\tilde{g} \in G$ such that $g \cdot x = \tilde{g} \cdot x$, therefore we could solve the DHSAP computing

$$\tilde{g} \cdot (h \cdot x) = (\tilde{g}h) \cdot x = (h\tilde{g}) \cdot x = h \cdot (\tilde{g} \cdot x) = h \cdot (g \cdot x) = (hg) \cdot x = (gh) \cdot x.$$

It is still an open question if the DHSAP and the SAP are equivalent, therefore all the attacks considered attempt to solve the Semigroup Action Problem.

Remark 2.27. If we are considering a semigroup G , we can always take $S = G$ and consider as a semigroup action just the semigroup operation. This is actually what we will do in the next section with $S = G = \mathbf{Knots}$.

Before we present our protocol, there is one more thing we need to introduce: cryptographic hash functions.

Definition 2.28. A *cryptographic hash function* is a deterministic and efficiently computable function $H: \{0, 1\}^* \rightarrow \{0, 1\}^\ell$ that maps inputs of arbitrary length to fixed-length outputs. It satisfies the following properties:

1. *Preimage resistance:* Given $y \in \{0, 1\}^\ell$, it is computationally infeasible to find an input x such that $H(x) = y$.
2. *Second preimage resistance:* Given an input x_1 , it is computationally infeasible to find $x_2 \neq x_1$ such that $H(x_1) = H(x_2)$.
3. *Collision resistance:* It is hard to find any two distinct inputs $x_1 \neq x_2$ such that $H(x_1) = H(x_2)$.

Another highly useful and typically essential property of a cryptographic hash function is the *Avalanche effect*: a small change in the input produces a significant and unpredictable change in the output.

In key exchange protocols, it is important that the final session key appears indistinguishable from a uniformly random string to any adversary. To achieve this, the cryptographic hash function used is typically required to behave as a *pseudorandom generator* (PRG). A PRG is a deterministic function that takes a secret key k and expands it into a longer output that is computationally indistinguishable from truly random bits. In other words, no efficient adversary can distinguish the output of the PRG from a uniformly random string with non-negligible advantage. This property plays a crucial role in proving indistinguishability, a fundamental property of secure key exchange, which is discussed in Appendix 2.

In simple terms, cryptographic hash functions are used to take input data and produce a fixed-size output that looks random and unpredictable. One important reason to use them is that even if your original output is not uniformly distributed, hashing it can spread the values evenly across the possible outputs.

3 Cryptosystem

The proposed cryptosystem is built by using the semigroup of oriented knots in the (Generalised Diffie-Hellman Key Exchange) Protocol 2.24 with the choices as in Remark 2.27. The security of the protocol is ensured by the fact that the general Problem 2.26 in our case translates to Problem 2.14, which is believed to be hard.

Protocol 3.1 (Knot-based Diffie-Hellman). All knot diagrams in this protocol are presented in PD notation, as described in Subsection 2.1.2. Let H be a cryptographic hash function with outputs in $\{0, 1\}^\ell$.

1. Alice and Bob publicly agree on a (randomly generated) alternating prime knot diagram K , a positive integer n and a finite type invariant V taking values in $\mathbb{Z}$.
2. Alice chooses a (randomly generated) alternating prime knot diagram A with at most n crossings, computes $A\#K$, complexifies the obtained knot diagram by applying a combination of the four complexification moves described in Subsection 2.1.2 and sends it to Bob. Her secret key is A .
3. Bob chooses a (randomly generated) alternating prime knot diagram B with at most n crossings, computes $B\#K$, complexifies the obtained knot diagram by applying a combination of the four complexification moves described in Subsection 2.1.2 and sends it to Alice. His secret key is B .
4. Alice computes $H(V(A\#(B\#K)))$.
5. Bob computes $H(V(B\#(A\#K)))$.

The secret common key is $H(V(A\#B\#K)) = H(V(B\#A\#K)) \in \{0, 1\}^\ell$.

There are two things about this protocol that are worth dwelling on. At first, there is the fact that we complexify the knot diagram $A\#K$ (respectively $B\#K$). The reason is that otherwise, it is very easy to see where the connected sum was made and we can decompose the knot by looking at its PD notation. The other thing that makes our protocol not quite a standard Diffie-Hellman protocol is that, in the end, we compute a knot invariant. The reason is that even though we know that Alice and Bob share the same knot $A\#B\#K = B\#A\#K$, they have very different presentations of that knot. Thus, the presentations themselves are useless for doing encryption, so we compute a knot invariant to get an integer, which is the same for both Alice and Bob.

The reason we have to apply the hash function at the end of the protocol arises from the experimental observation that finite type invariant outputs are not uniformly distributed, but tend to be concentrated near zero.

Notice that the bottleneck of the protocol is the computation of the finite type invariant, which requires $\mathcal{O}(c^{\frac{m}{2}})$ operations, where c is the number of crossings of the final knots $A\#B\#K$ and $B\#A\#K$ (recall that in their description these two knots can have a different number of crossings) and m is the degree of the chosen finite type invariant V .

Key size. Assume that in the complexification phase of the knot diagram, both Alice and Bob apply complexification moves until they obtain an equivalent knot diagram with at most $2n$ crossings.

In order to describe a knot diagram with n crossings, we need all the positive integers from 1 to $2n$, just to enumerate all the edges. Following the encoding procedure, we have to write each integer between 1 and $2n$ twice in the string of integers, since each edge is related to two crossings. Recall that we need at most (up to a constant)

$$\sum_{i=1}^{\lfloor \log_2(n) \rfloor} i2^{(i-1)} - 1 = 2^{\lfloor \log_2(n) \rfloor} \lfloor \log_2(n) \rfloor - 2^{\lfloor \log_2(n) \rfloor} \simeq n(\log_2(n) - 1) \text{ bits}$$

to describe all the integers from 1 to n .

Summing up, given a knot diagram with n crossings, we need roughly $2n(\log_2(2n) - 1)$ bits to describe all the integers from 1 to $2n$ that represent the edges and, since we need to use each of them twice, we need roughly $4n(\log_2(2n) - 1)$ bits to describe Alice's private key and Bob's private key respectively.

Remark 3.2. Since the bottleneck of the protocol is the computation of the finite type invariant, we would like to choose an invariant of degree as small as possible. But we need it to be of degree large enough to ensure that a brute force attack is not possible. In fact, one can decide to compute not just one finite type invariant, but several different ones, all of small degree, since the complexity is essentially the same.

4 Cryptanalysis

In the following, we analyse the security and the efficiency of the proposed protocol. We explain how other invariants than the ones of finite type fail and discuss the choice of parameters.

4.1 Security analysis

We discuss several possible attacks on the proposed protocol and either explain why they fail or give our opinion on how likely they would work to break the protocol.

Notice that the underlying mathematical problem is the following one.

Problem 4.1. Given V a finite type invariant and the knots K , $A\#K$ and $B\#K$, find $V(A\#B\#K)$.

Notice that if, for example, we would have chosen the Jones polynomial [23] as our knot invariant the protocol would break. This is because the Jones polynomial J is multiplicative with respect to the connected sum, so one can compute $J(A) = \frac{J(A\#K)}{J(K)}$ or $J(B) = \frac{J(B\#K)}{J(K)}$, which leads to $J(A\#B\#K) = J(A)J(B\#K)$ or $J(A\#B\#K) = J(B)J(A\#K)$. Thus, we want to avoid knot invariants which have a connected sum formula. We don't know of any such general formula for finite type invariants. But it may be (and we actually know it for some of them as discussed in Section 4.3) that the specific finite type invariant we choose has such a formula. The strength of the proposed protocol however then lies in the fact that we are free to choose another finite type invariant for which we know no connected sum formula.

The study of an attack for a given finite type invariant is far from exhaustive. It is possible that some of them allow ad hoc attacks and should therefore be excluded.

There is a related mathematical problem, which is the SAP 2.26 on **Knots**.

Problem 4.2. Given two knots K and K' , find a knot A (if it exists) such that $K' = A \# K$.

Notice that, solving the previous problem allows an attacker to solve the underlying problem 4.1. Indeed, given K and $A \# K$ as in the protocol, if he finds A , since he also knows $B \# K$, he can compute $A \# B \# K$; at this point, he only has to calculate the finite type invariant of $A \# B \# K$ in order to get the secret shared key.

In the following, we discuss several possible attacks on the SAP on **Knots**. We divide them into two families: those related to the generic SAP and those that explicitly use Knot Theory.

4.1.1 Generic attacks for the semigroup action problem

Here we analyse the generic attacks on the SAP described in [33] and [32].

Feasibility of the SAP. In general, when we try to solve the SAP, we are looking for an $h \in G$ such that $g \cdot x = h \cdot x$, not necessarily for the specific g . For this reason, we are interested in the following set

$$G_{x,g} = \{h \in G \mid h \cdot x = g \cdot x\}.$$

The parameters G , S and x need to be chosen in such a way that $|G_{x,g}|$ is small with respect to $|G|$. In our case we have $G = S = \mathbf{Knots}$, $x = K \in S$ and the action is just the connected sum. Thanks to Proposition 2.7, we know that, given K and $K' \# K$, the knot K' is unique, therefore $G_{K,K'} = \{K'\}$ and $|G_{K,K'}| = 1$ for all $K, K' \in \mathbf{Knots}$.

Structure of the semigroup. Various attacks on DH realised through a group action are known and it is therefore legitimate to ask whether some of them can also be applied or generalised to semigroup actions. First, notice that we can partition the semigroup G , following the notation in [32], as $G = G_0 \cup G_1$, where

$$G_1 = \{g \in G \mid g^{-1} \text{ exists}\} \quad \text{and} \quad G_0 = G \setminus G_1.$$

Practically, if G has a large subgroup, it can be a weakness in the protocol. Indeed, we can try to solve the SAP in G_0 by brute force, with an exhaustive search. If we found no solutions, then we can restrict the SAP to G_1 , which is a group. There all the attacks that we know for groups are applicable. In our case, this strategy does not apply, because $G_1 = \{\mathcal{U}\}$, thanks to Proposition 2.6.

4.1.2 Knot theoretic attacks

Attacking the basic structure and the complexification phase. In our proposed protocol there are three steps which need our attention. Taking the connected sum, complexifying the diagram, and the evaluation via a knot invariant. We have already discussed above, at the beginning of the section, the choice of the finite type invariant. The connected sum poses no problems for security. Provided that the knot is in a general position, it is commonly accepted that decomposition as in Problem 2.14 is very hard. However, if you just take the connected sum and leave the knot as it is, it is easy to decompose it by looking at its PD notation. So the question becomes, whether the algorithm using a combination of the four complexification moves achieves a position for the knot which is random enough that one cannot decompose the knot easily. This is still an open question. What has been investigated is that using only random Reidemeister moves is not enough as in all examples, the knot was easily recognizable.

Brute force attack. Of course, the first attack that one can always try is the brute force attack, which means that the attacker has to compute $A' \# K$, for all $A' \in \mathbf{Knots}$ with at most n crossings, until he finds the correct one, which is $A \# K$. But recall that he doesn't have a way to compare directly $A' \# K$ with $A \# K$, since the second one is complexified in the protocol and Problem 2.12 is hard. The best thing that he could do is computing a fixed invariant of $A' \# K$ and $A \# K$ and check if it is the same. In general, in order to avoid brute force attacks, it is sufficient to set the parameters of the cryptosystem appropriately. We will discuss the choice of parameters in Subsection 4.4. But recall that we don't have a computable complete invariant, so it could happen that more than one knot A' satisfy that $A' \# K$ and $A \# K$ have the same fixed invariant. In that case, the attacker could change the invariant and compute it only for previously acceptable candidates or for each of them (if they are few) he can compute the finite type invariant of $A' \# B \# K$ and check which one works as the cryptographic key.

Actually, since the cryptographic key is the hash of an integer given by the finite type invariant of the final knot, an attacker can also apply a brute force attack to this value. The unique upper bound that we have in general is given in Corollary A.4. However, we do not expect a finite type invariant to have a uniform distribution, therefore it is possible that some integers are more likely than others. Some experiments showed that some of them, at least of small degree, look more distributed close to 0. One solution to this problem is to compute more than one finite-type invariant.

Problem with small knots. One thing to keep in mind is that knots with up to around 15 crossings can be taken apart with knot invariants. This forces us to use bigger knots. Otherwise one could compare the invariants of the public knots to a table of knots with their invariants and try the corresponding knot. The number of (alternating prime) knots however increases very fast as seen in Proposition 2.10, which means that this procedure is not possible for bigger knots.

Attack using invariants with a connected sum formula. If the knots chosen by Alice and Bob in the protocol are sums of smaller knots, there is the attack described below. In Subsection 4.3.2 we will see that a lot of knot invariants have a connected sum formula. Take for example the Alexander polynomial Δ . It satisfies $\frac{\Delta(A \# K)}{\Delta(K)} = \Delta(A)$. Hence, one can get the invariant of the secret knot. If this knot is composed of small knots, one can factor $\Delta(A)$ and only consider prime knots with Alexander polynomial equal to one of the factors (including the constant polynomial 1). This will lead to a combinatorial problem that is more approachable, given that one can compute the Alexander polynomial on all prime knots with a given crossing number. This attack also generalizes to other knot invariants with a connected sum formula. This attack is the reason why we work with prime knots in the protocol.

Attack for braid group cryptography. Knots and braids (see Figure 13) are intimately related by the Alexander and Markov Theorem proven in [10]. The basic idea is that closing up a braid yields a knot (or a link). But the group structure on braids is not related to the semigroup structure on knots. There are cryptographic protocols involving braid groups (see [26, 19]) which are now broken. They were successfully attacked by using the faithful Lawrence representation of the braid group, where one could transfer the underlying problem into the world of matrices (compare [29], [9], and [27]). Contrary to braid groups, we know no reasonable representation of the semigroup of knots, so attacks of this form are not available.

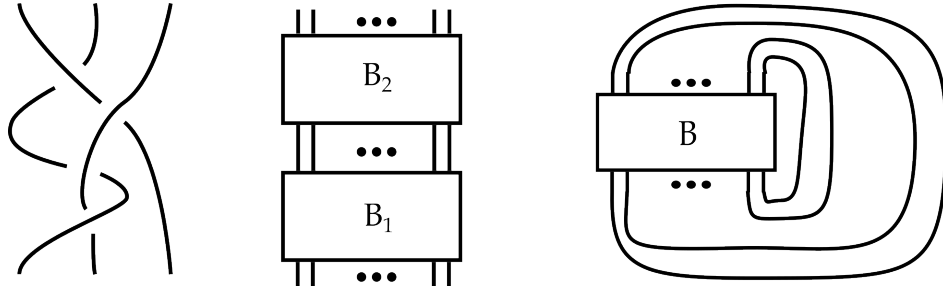


Figure 13: An example of a braid with three strands, the group structure of the braid group given by stacking and the closure of a braid.

Knots seen as graphs. In general, a linear representation of the problem could be a weakness; indeed, if we can represent the action as a matrix action on a vector space, then the SAP may be solved easily. One can think of a knot as a graph (keeping track of over- or undercrossing) and to a graph we can associate the adjacency matrix. However, we can't represent the connected sum as a matrix action. Moreover, if we consider a complicated knot and the adjacency matrix associated to it, we can't obtain enough information to understand which knot it is.

We are therefore unable to represent the action as a linear one, or at least we are unable to do so by exploiting the one related to braid groups nor the point of view of graphs. This obviously does not represent an exhaustive study of all possible ways in which one can try to represent the connected sum as a linear action.

4.2 Efficiency analysis

We show here that our proposed protocol can be ran in polynomial time, depending on the crossing number. We have three knots involved K , A , and B with a number of crossings c_K , c_A , and c_B , respectively. Taking the connected sum is an easy operation. We trust that obfuscating the diagram is not very costly as well, since all the moves are easy 3D-moves and the application of Reidemeister moves in terms of the PD notation is efficient. This leaves the computation of the finite type invariant which uses $\sim (2c_K + 2c_A + c_B)^{\frac{m}{2}}$ steps for a finite type invariant of degree m , thanks to Theorem 2.19.

4.3 Other invariants

In principal, one can choose any knot invariant for the protocol to work. Maybe, there is a knot invariant that we do not know and that fits the protocol better then the finite type ones. However there are some features of certain knot invariants that make the protocol very slow or even attackable. We discuss them here.

4.3.1 Computability

First, there is the problem of computability. Low-dimensional topology is notorious for having complicated (and powerful) constructions which are hard to calculate. This rules out basically all

knot homologies like the Khovanov homology [25] and the knot Floer homology [38], as well as any invariant coming from them, like the Rassmussen invariant [41], the Υ [39] and the τ invariant [37]. Even the Jones polynomial is too costly to calculate, since its computation time grows exponentially in the number of crossings [24]. The genus, braid index and similar invariants have no algorithms to calculate them at all [42].

4.3.2 Connected sum formula

As already explained before, the protocol is attackable when the used invariant has a reasonably nice connected sum formula. Then the attacker does not have to decompose the knot itself, but rather has to find the invariant of the individual knots. We list here which invariants this excludes. The HOMFLY, Jones, and Alexander knot polynomials [30] are all multiplicative with respect to the connected sum and are therefore immediately excluded. Even the “Polynomial time knot polynomial” of Bar Natan [8] has a nice enough connected sum formula that can be tackled. The signature is additive [42] and the number of 3-colorings is multiplicative [42].

We show here how even some finite type invariants fail. Let us consider f_m the m -th coefficient of the Conway polynomial C considered in Example 2.16, i.e. $C(K) = \sum_{m \in \mathbb{N}} f_m z^m$. The polynomial itself is multiplicative, i.e. $C(K \# A) = C(K)C(A)$. While the finite type invariant f_m is not multiplicative, it has the following formula

$$f_m(K \# A) = \sum_{i=0}^m f_i(K) f_{m-i}(A).$$

Thus, knowing $f_i(K)$ and $f_i(K \# A)$ for $0 \leq i \leq m$ (which are computable in not much more time than $f_m(K \# A)$) allows an attacker to solve for $f_i(A)$ (and similarly $f_i(B)$), which is enough to compute $f_m(K \# A \# B)$.

With exactly the same reasoning, one can also exclude the finite type invariants given by the m -th coefficient of $J(e^x)$ (where J is the Jones polynomial) seen as a formal power series in x . This tells us that we have to choose our finite type invariant carefully enough to avoid these specific ones. But there are plenty of other finite type invariants.

4.4 Choice of Parameters

The only parameter that we have to choose is the number of crossings n . As far as we know, the best attack is the brute force attack described above in this chapter, since the knots chosen by Alice and Bob are prime.

So we need to look at the number of alternating prime knots with at most n crossings. Looking at Proposition 2.10, we expect the number of prime alternating knots to grow exponentially, and we believe that choosing n on the order of 10^3 should be sufficient to achieve a 128-bit security level - meaning that an adversary has to perform at least 2^{128} operations to break the protocol.

Since the proposition provides only bounds in terms of $\liminf$, we cannot at this stage give a sharper estimate without having access to an explicit implementation. In future work, one of our first objectives will be to determine more precisely the admissible range of n as the security parameter λ varies (above $\lambda = 128$).

5 Conclusion and further development

In the paper, we propose a new key exchange protocol based on the Generalised Diffie-Hellman Key Exchange Protocol. We use the semigroup action given by

$$\begin{aligned} \# : \mathbf{Knots} \times \mathbf{Knots} &\rightarrow \mathbf{Knots} \\ (K_1, K_2) &\mapsto K_1 \# K_2, \end{aligned}$$

where $\#$ represents the connected sum of two knots.

Since in our protocol Alice and Bob get the same connected sum of three knots, but represented in two different ways, in the last step we need to compute an invariant of this knot in order to obtain the same shared secret key. We studied the different possibilities and concluded that the best choice are finite type invariants, since they give us a bounded positive integer, they can be computed in polynomial time and they do not admit a connected sum formula.

Furthermore, after the cryptanalysis, the best attack is a sort of brute force attack.

A security proof against a passive adversary is provided in [Appendix B](#).

Open problems. The following problems are still open and are interesting for a future work:

- A much deeper study of finite type invariants is needed. We should study each of them for degree $m \geq 3$, to understand which are the most suitable ones. It could be that some of them admit a connected sum formula, which means that we have to avoid them. This analysis also is needed to understand which degree m is most suitable. This choice is fundamental, since the computational complexity of a finite type invariant of degree m is exactly $\mathcal{O}(c^{\frac{m}{2}})$, where c is the number of crossings of the knot. As we said, this computation is the bottleneck of the protocol.
- Our complexification method is really ad hoc. It is crucial to investigate how effectively this method generates randomness in knots. Specifically, we need to determine whether it achieves a sufficient level of randomness.
- An implementation, specifically concerning the computation of low degree finite type invariants, is in progress (thanks to Chris Monico).
- Finally, in [Appendix B](#), we provide a security proof against passive adversaries. Our near future plans include formalising a Key Encapsulation Mechanism (KEM) based on the proposed key exchange protocol, as well as providing a more rigorous security proof that considers active adversaries.

A Finite type invariants are computable in polynomial time

The aim of this appendix is to show how one can compute finite type invariants in polynomial time. Additionally, we will see a way to construct any finite type invariant.

A.1 Gauss diagrams

We first consider Gauss diagrams. They are closely related to PD notation in the sense that the former are a visualisation of the latter.

Start with a knot diagram with n crossings. Label its crossings starting at a basepoint on the knot, following the orientation, with the numbers 1 to $2n$ such that every crossing has two numbers assigned to it. To construct the Gauss diagram, we draw a circle with $2n$ dots with labels from 1 to $2n$ and connect the points belonging to the same crossing with a line and decorate it using the sign of the crossing. In addition, we give an orientation to the line to indicate which strand passed over the other one. See an example in Figure 14. We can consider arbitrary Gauss diagrams which do not need to come from knots. In fact, there are examples of Gauss diagrams that cannot be realised as knot in $\mathbb{R}^3$ without introducing singularities.

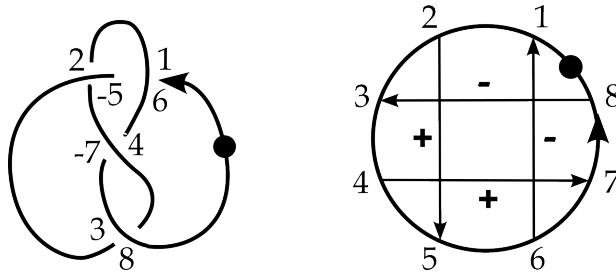


Figure 14: The Gauss diagram for the Figure Eight knot.

Definition A.1. The *space of Gauss diagrams* $\mathcal{D}$ is the $\mathbb{Q}$ -vector space spanned by all Gauss diagrams.

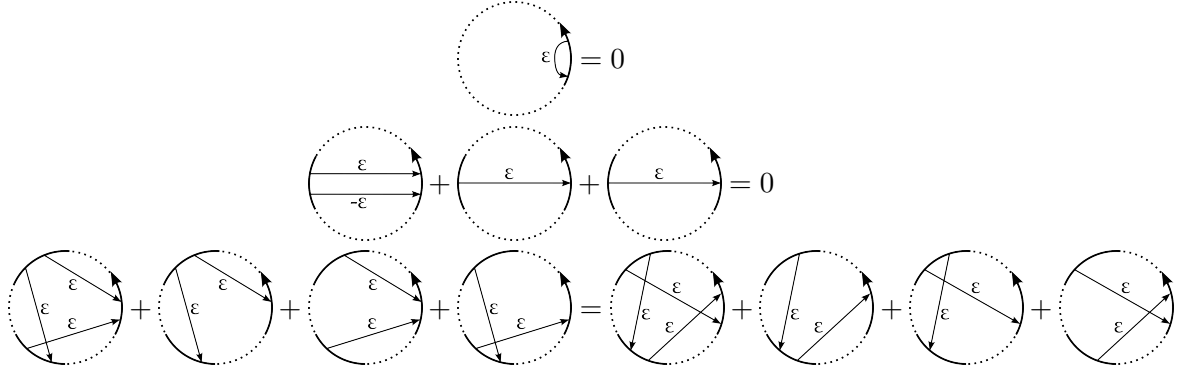
A *Gauss subdiagram* D' of D is a diagram in which some arcs of D have been deleted and we write $D' \subset D$. We can define the following endomorphism:

$$\begin{aligned} \phi: \mathcal{D} &\longrightarrow \mathcal{D} \\ D &\mapsto \sum_{D_i \subset D} D_i . \end{aligned}$$

A.2 The Polyak space and the proof

Clearly, ϕ doesn't yield a knot invariant. Just introducing more crossings in a diagram changes its image under ϕ . To make ϕ an invariant, we need to introduce relations in the space of Gauss diagrams $\mathcal{D}$.

Definition A.2. The *Polyak space* $\mathcal{P}$ is defined to be $\mathcal{A}$ with the following relations:



The space $\mathcal{P}_m$ is obtained by setting all diagrams with more than m arcs to zero. It yields a finite dimensional vector space. The restriction of ϕ to $\mathcal{P}_m$ is denoted by ϕ_m . It is a universal finite type invariant in the sense of the next theorem.

Theorem A.3 ([20]). *V is a finite type invariant of degree m (with values in $\mathbb{Q}$) if and only if $V = f \circ \phi_m$, where $f: \mathcal{P}_m \rightarrow \mathbb{Q}$ is a linear functional.*

From this we can prove Corollary 2.18.

Proof of Corollary 2.18. To compute the map ϕ_m one needs to compute all $\sum_{i=1}^m \binom{c}{i}$ Gauss subdiagrams, which requires the stated polynomial amount of operations of Gauss diagrams:

$$\sum_{i=1}^m \binom{c}{i} \sim \sum_{i=1}^m c^i \sim c^m.$$

The evaluation of these subdiagrams via f doesn't add computational cost. □

Note that the previous proof allows us to construct any finite type invariant and thus, can be seen as a blueprint to construct finite type invariants. One can calculate a basis of the finite dimensional vector space $\mathcal{P}_m$ and then choose any linear functional on this basis. Note that the finite type invariant in principle takes values in $\mathbb{Q}$. It is more convenient to have an invariant that takes values in $\mathbb{Z}$, which is easily obtained by scaling the invariant by the least common multiple of the divisors of the values on the basis of Gauss subdiagrams, such that they evaluate to integers. The following corollary is immediate.

Corollary A.4. *Let the knot K have a diagram with c crossings and let V be a finite type invariant of degree m . Define $M := \max_{D \text{ Gauss diagram}} V(D)$. We have the following bound:*

$$|V(K)| \leq M \cdot 2^c.$$

B Security proof

In this appendix, we formalise our security assumptions and present a proof of semantic security against *passive* adversaries — those limited to eavesdropping without actively engaging in the communication. We also assume that the two parties involved in the key exchange are honest.

One of the fundamental security properties that a key exchange protocol must achieve is *indistinguishability*. Informally, this property requires that an adversary cannot distinguish between a legitimately established shared secret key and a uniformly random element. In our setting, we establish indistinguishability under the assumption that the Decisional Knot Diffie–Hellman (DKDH) problem, defined below, is computationally hard.

Notation. We recall the notation $\text{APK}(n)$ to indicate the set of alternating prime knots such that the minimal representation has at most n crossings. In the following, we will indicate with $\sim : \text{APK}(n) \rightarrow \text{APK}(n)$ the complexification algorithm described in Subsection 2.1.2. We define also the set $\mathcal{R} = \{A\#B \mid A, B \in \text{APK}(n)\}$. Finally, we write $K \sim K'$ to indicate that K and K' represent the same knot, i.e. they are in the same equivalence class.

Problem B.1 (Decisional Knot Diffie–Hellman Problem). Let $K, A, B \in \text{APK}(n)$. Given $K, \widetilde{A\#K}, \widetilde{B\#K}$ and $\widetilde{R\#K}$, establish if $R \sim A\#B$ or $R \in \mathcal{R}$ random.

Notice that Problem B.1 strictly depends on the change of representation considered.

In the following we will refer to Protocol 3.1 with KKE (Knot Key Exchange).

Definition B.2. Let the game IND – DKDH be defined as in Table 3. For a passive adversary $\mathcal{A}$, define the adversary’s *advantage* as

$$\begin{aligned} \text{Adv}^{\text{IND-DKDH}}(\mathcal{A}) = & |\mathbb{P}[\mathcal{A}(\widetilde{A\#K}, \widetilde{B\#K}, s) = 0 \mid s = H(V(A\#B\#K))]| - \\ & |\mathbb{P}[\mathcal{A}(\widetilde{A\#K}, \widetilde{B\#K}, s) = 1 \mid s \xleftarrow{\$} \{0, 1\}^\ell]| \end{aligned}$$

The key exchange protocol **KKE** is *DKDH-indistinguishable* or *IND-DKDH secure* if, for every PPT passive adversary $\mathcal{A}$, the advantage $\text{Adv}^{\text{IND-DKDH}}(\mathcal{A})$ is negligible.

We will now prove that under the assumption that the DKDH Problem is hard, our protocol is DKDH-indistinguishable.

Theorem B.3. Assume H is a cryptographic hash function which is a PRG and that the chosen finite type invariant V is collision-resistant. Assume that the DKDH Problem is hard with respect to our complexification algorithm $\sim$. Then the KKE Protocol is IND-DKDH secure.

Proof. Assume there exists a PPT adversary $\mathcal{A}$ such that $\text{Adv}^{\text{IND-DKDH}}(\mathcal{A})$ is non-negligible. We will construct a PPT adversary $\mathcal{B}$ able to solve the DKDH problem with non-negligible probability.

More precisely, we want to prove that

$$\begin{aligned} \text{Adv}^{\text{DKDH}}(\mathcal{B}) = & |\mathbb{P}[\mathcal{B}(K, \widetilde{A\#K}, \widetilde{B\#K}, T) = 0 \mid T \sim A\#B\#K]| - \\ & |\mathbb{P}[\mathcal{B}(K, \widetilde{A\#K}, \widetilde{B\#K}, T) = 1 \mid T \sim R\#K, R \xleftarrow{\$} \mathcal{R}]| \end{aligned}$$

is non-negligible.

Challenger $\mathcal{C}(n, K, V, H)$	Adversary $\mathcal{A}(n, K, V, H)$
$A, B \leftarrow \text{APK}(n);$	
Computes $\widetilde{A\#K}, \widetilde{B\#K};$	
	$\xrightarrow{\widetilde{A\#K}, \widetilde{B\#K}}$
Computes $v_0 := V(A\#B\#K);$	
Computes $h_0 := H(v_0);$	
$b \xleftarrow{\$} \{0, 1\};$	
if $b = 0$	
$s := h_0;$	
if $b = 1$	
$s \xleftarrow{\$} \{0, 1\}^\ell;$	
	$\xrightarrow{s}$
	Outputs $b' \in \{0, 1\}$

Table 3: The indistinguishability game related to the KKE Protocol, where n is a positive integer, $K \in \text{APK}(n)$ is the public base knot, V is the finite type invariant and H is the cryptographic hash function.

The strategy of $\mathcal{B}$ is as follows. It gets as input $K, A, B \in \text{APK}(n)$ and a knot diagram T such that $T = \widetilde{R\#K}$ with $R \sim A\#B$ or $R \in \mathcal{R}$ random. It computes $v = V(T)$ and $s = H(V(T))$, where V is the finite type invariant of the protocol. Now, it invokes $\mathcal{A}$, providing it with $(\widetilde{A\#K}, \widetilde{B\#K}, s)$. It receives $\mathcal{A}$'s guess $b' \in \{0, 1\}$ with $b' = 0$ if $\mathcal{A}$ believes that $s = H(V(A\#B\#K))$ or $b' = 1$ if $\mathcal{A}$ believes that s is random. The bit b' is $\mathcal{B}$'s output.

Let $p_{\text{real}} = \mathbb{P}[\mathcal{A}(\widetilde{A\#K}, \widetilde{B\#K}, s) = 0 \mid s = H(V(A\#B\#K))]$ and $p_{\text{random}} = \mathbb{P}[\mathcal{A}(\widetilde{A\#K}, \widetilde{B\#K}, s) = 1 \mid s \xleftarrow{\$} \{0, 1\}^\ell]$.

If $T \sim A\#B\#K$, then $v = V(A\#B\#K)$ and $s = H(V(A\#B\#K))$, therefore

$$\mathbb{P}[\mathcal{B} \text{ outputs } 0] = \mathbb{P}[\mathcal{A} \text{ outputs } 0] = p_{\text{real}}.$$

If $T \sim R\#K$ with $R \in \mathcal{R}$ random, then $v = V(R\#K)$ and $s = H(V(R\#K))$. It is now necessary to study some problematic situations and the probability with which they may occur. It can happen that $V(R\#K) \neq V(A\#B\#K)$, but $H(V(R\#K)) = H(V(A\#B\#K))$. In this case, $\mathcal{A}$ outputs 1 even if $T \not\sim A\#B\#K$. Thanks to the fact that H is a cryptographic hash function, this scenario occurs with a negligible probability that we indicate with $p_{\text{coll_hash}}$.

It can happen that $R\#K \not\sim A\#B\#K$, but $V(R\#K) = V(A\#B\#K)$. Also in this case, $\mathcal{A}$ outputs 1 even if $T \not\sim A\#B\#K$. Thanks to the hypothesis that V is collision-resistant, this scenario occurs with a negligible probability that we indicate with $p_{\text{coll_inv}}$.

In summary, we get

$$\mathbb{P}[\mathcal{B} \text{ outputs } 1] = \mathbb{P}[\mathcal{A} \text{ outputs } 1] = p_{\text{random}} - p_{\text{coll_hash}} - p_{\text{coll_inv}}.$$

Therefore

$$\text{Adv}^{\text{DKDH}}(\mathcal{B}) = |\text{Adv}^{\text{IND-DKDH}}(\mathcal{A}) - p_{\text{coll_hash}} - p_{\text{coll_inv}}|;$$

since $p_{\text{coll_hash}}$ and $p_{\text{coll_inv}}$ are negligible by assumption, and $\text{Adv}^{\text{IND-DKDH}}(\mathcal{A})$ is not, we conclude that $\text{Adv}^{\text{DKDH}}(\mathcal{B})$ is not-negligible too. $\square$

Remark B.4. The assumption that V is collision-resistant can seem a bit too strong and optimistic. Rather than relying on a single finite type invariant, it is more natural to consider several of them of low degree. Since the computational complexity is essentially the same for computing one finite type invariant of degree at most d as for computing all such invariants, this approach does not introduce additional cost. In view of Conjecture 2.17, we expect that combining multiple finite type invariants will make it possible to obtain a collision-resistant function, or at least one that approximates this property sufficiently.

References

- [1] Ilya Alekseev, Anatolii Moiseevich Vershik, and Andrei Valer’evich Malyutin. On the growth of the number of prime knots. *Algebra i Analiz*, 36(1):17–39, 2024.
- [2] James W Alexander. Topological Invariants of Knots and Links. *Transactions of the American Mathematical Society*, 30(2):275–306, 1928.
- [3] Z Amir-Khosravi and S Sankaran. Vascalc—a vassiliev invariants calculator, electronic document tree.
- [4] Iris Anshel, Michael Anshel, and Dorian Goldfeld. An algebraic method for public-key cryptography. *Mathematical Research Letters*, 6(3):287–291, 1999.
- [5] Dror Bar-Natan. On the Vassiliev knot invariants. *Topology*, 34(2):423–472, 1995.
- [6] Dror Bar-Natan, Itai Bar-Natan, Iva Halacheva, and Nancy Scherich. Computing finite type invariants efficiently. *arXiv preprint arXiv:2408.15942*, 2024.
- [7] Dror Bar-Natan, Scott Morrison, and et al. The Knot Atlas. <http://katlas.org>.
- [8] Dror Bar-Natan and Roland van der Veen. A polynomial time knot polynomial. *Proceedings of the American Mathematical Society*, 147(1):377–397, 2019.
- [9] Stephen Bigelow. Braid groups are linear. *Journal of the American Mathematical Society*, 14(2):471–486, 2001.
- [10] Joan S. Birman and Tara E. Brendle. Braids: a survey. *Handbook of knot theory*, pages 19–103, 2005.
- [11] Benjamin A Burton. Maximal admissible faces and asymptotic bounds for the normal surface solution space. *Journal of Combinatorial Theory, Series A*, 118(4):1410–1435, 2011.
- [12] Benjamin A Burton. The Pachner graph and the simplification of 3-sphere triangulations. In *Proceedings of the twenty-seventh annual symposium on Computational geometry*, pages 153–162, 2011.

- [13] Wouter Castryck, Tanja Lange, Chloe Martindale, Lorenz Panny, and Joost Renes. Csidh: an efficient post-quantum commutative group action. In *Advances in Cryptology—ASIACRYPT 2018: 24th International Conference on the Theory and Application of Cryptology and Information Security, Brisbane, QLD, Australia, December 2–6, 2018, Proceedings, Part III 24*, pages 395–427. Springer, 2018.
- [14] John H Conway. An enumeration of knots and links, and some of their algebraic properties. In *Computational problems in abstract algebra*, pages 329–358. Elsevier, 1970.
- [15] Joan Daemen and Vincent Rijmen. AES proposal: Rijndael. 1999.
- [16] Whitfield Diffie and Martin Hellman. New directions in cryptography (1976). *IEEE Trans. Inform. Theory*, 22:644–654, 1976.
- [17] Ivan Dynnikov. Arc-presentations of links: monotonic simplification. *Fundamenta Mathematicae*, 1(190):29–76, 2006.
- [18] Michael H Freedman and Frank Quinn. Topology of 4-manifolds (pms-39), volume 39. 49, 2014.
- [19] David Garber. Braid group cryptography. *Braids: Introductory lectures on braids, configurations and their applications*, pages 329–403, 2010.
- [20] Mikhail Goussarov, Michael Polyak, and Oleg Viro. Finite-type invariants of classical and virtual knots. *Topology*, 39(5):1045–1068, 2000.
- [21] Joel Hass, Jeffrey C Lagarias, and Nicholas Pippenger. The computational complexity of knot and link problems. *Journal of the ACM (JACM)*, 46(2):185–211, 1999.
- [22] Daemen Joan and Rijmen Vincent. The design of rijndael: AES-the advanced encryption standard. *Information Security and Cryptography*, 2002.
- [23] Vaughan FR Jones. A polynomial invariant for knots via von Neumann algebras. 1985.
- [24] Louis H Kauffman. State models and the Jones polynomial. *Topology*, 26(3):395–407, 1987.
- [25] Mikhail Khovanov. A categorification of the Jones polynomial. 2000.
- [26] Ki Hyoung Ko, Sang Jin Lee, Jung Hee Cheon, Jae Woo Han, Ju-sung Kang, and Choonsik Park. New public-key cryptosystem using braid groups. In *Advances in Cryptology—CRYPTO 2000: 20th Annual International Cryptology Conference Santa Barbara, California, USA, August 20–24, 2000 Proceedings 20*, pages 166–183. Springer, 2000.
- [27] Daan Krammer. Braid groups are linear. *Annals of Mathematics*, pages 131–156, 2002.
- [28] Marc Lackenby. A polynomial upper bound on Reidemeister moves. *Annals of Mathematics*, pages 491–564, 2015.
- [29] Ruth J Lawrence. Homological representations of the Hecke algebra. *Communications in mathematical physics*, 135:141–191, 1990.

- [30] William BR Lickorish and Kenneth C Millett. The new polynomial invariants of knots and links. *Mathematics Magazine*, 61(1):3–23, 1988.
- [31] Annalisa Marzuoli and Giandomenico Palumbo. Post quantum cryptography from mutant prime knots. *International Journal of Geometric Methods in Modern Physics*, 8(07):1571–1581, 2011.
- [32] Gérard Maze. *Algebraic Methods for constructing One-Way Trapdoor Functions*. University of Notre Dame, 2003.
- [33] Gérard Maze, Chris Monico, and Joachim Rosenthal. Public Key cryptography based on semi-group actions. *Adv. in Math. of Communications* 1.4, pages 489–507, 2007.
- [34] William Menasco. Closed incompressible surfaces in alternating knot and link complements. *Topology*, 23(1):37–44, 1984.
- [35] Christopher Monico. *Semirings and semigroup actions in public-key cryptography*. University of Notre Dame, 2002.
- [36] Kunio Murasugi. Jones polynomials and classical conjectures in knot theory. *Topology*, 26(2):187–194, 1987.
- [37] Peter Ozsváth and Zoltán Szabó. Knot Floer homology and the four-ball genus. *Geometry & Topology*, 7(2):615–639, 2003.
- [38] Peter Ozsváth and Zoltán Szabó. An overview of knot Floer homology. *arXiv preprint arXiv:1706.07729*, 2017.
- [39] Peter S Ozsváth, András I Stipsicz, and Zoltán Szabó. Concordance homomorphisms from knot Floer homology. *Advances in Mathematics*, 315:366–426, 2017.
- [40] John M Pollard. Monte Carlo methods for index computation (mod p). *Mathematics of computation*, 32(143):918–924, 1978.
- [41] Jacob Rasmussen. Khovanov homology and the slice genus. *Inventiones mathematicae*, 182(2):419–447, 2010.
- [42] Justin Roberts. Knots knots. *Lectures from Edinburgh Course Maths*, 415, 1999.
- [43] Daniel Shanks. Class number, a theory of factorization, and genera. In *Proc. Symp. Math. Soc.*, 1971, volume 20, pages 415–440, 1971.
- [44] Edward Witten. Quantum field theory and the Jones polynomial. *Communications in Mathematical Physics*, 121(3):351–399, 1989.
- [45] Marc Zucker. *Studies in cryptological combinatorics*. City University of New York, 2005.